

LLM04 - Supply Chain Attack

Key Security Controls | From trusted packages to production assurance

SECURITY OBJECTIVE

Trust and verify every component across the software, AI model, data, build, release, and runtime supply chain.

01

Approved Internal Package Registry

FOCUS: PACKAGE INTAKE & DEPENDENCY CONFUSION

Use a trusted internal registry containing only approved and security-validated packages. This prevents malicious or unapproved public packages from entering applications through dependency-confusion attacks.

Example: JFrog Artifactory

02

Exact Versions & Hash-Locked Dependencies

FOCUS: REPRODUCIBLE BUILDS

Lock dependencies to specific versions and verified hashes, ensuring every build uses exactly the same trusted software components.

Simple idea: Same code + Same dependencies = Same trusted build

03

Complete SBOM

FOCUS: DEPENDENCY VISIBILITY

Maintain an SBOM covering both direct and transitive dependencies, providing complete visibility into the software components inside the application.

Direct dependency -> Transitive dependency -> Full visibility

04

Approved & Signed Model Artifacts

FOCUS: MODEL SUPPLY CHAIN

Use only trusted, approved, and digitally signed AI models. Signing helps verify that a model is authentic and has not been tampered with.

Trusted source + Digital signature = Verified model

05

RAG Source Provenance & Sanitization

FOCUS: DATA SUPPLY CHAIN

Ensure RAG data comes from trusted and traceable sources, and sanitize it before ingestion to reduce poisoned, malicious, or manipulated content reaching the AI.

Trusted source + Sanitization = Safer AI context

06

Least-Privilege Plugin Policy

FOCUS: AGENT & TOOL GOVERNANCE

Give AI agents and plugins only the minimum tools and permissions required for their tasks. This limits the impact of compromised or malicious tools.

Minimum access -> Reduced blast radius

LLM04 - Supply Chain Attack

Pipeline, build, release and production controls

07

OIDC Short-Lived CI/CD Credentials

FOCUS: PIPELINE SECURITY

Use temporary, short-lived credentials for CI/CD pipelines instead of storing long-term secrets. Even if exposed, their usefulness to an attacker is limited.

Temporary credentials -> Lower secret exposure

08

Signed Container Images Pinned by Digest

FOCUS: CONTAINER INTEGRITY

Deploy the exact approved and verified container image using its immutable digest. This prevents modified, substituted, or unauthorized images from reaching production.

Signature + Digest = Exact verified image

09

Reviewed & Hermetic Build Scripts

FOCUS: BUILD INTEGRITY

Build software using reviewed scripts in a controlled environment with fixed tools and dependencies, preventing unexpected external components from entering the build.

Hermetic build: Controlled environment + Fixed dependencies + Fixed tools

10

SLSA-Style Provenance

FOCUS: RELEASE TRACEABILITY

SLSA = Supply-chain Levels for Software Artifacts. Think of provenance as a digital birth certificate for a release - showing where it came from, how it was built, and what was used.

Provenance -> Verify and trace every release

11

Runtime Integrity & Drift Monitoring

FOCUS: PRODUCTION ASSURANCE

Continuously verify that production is running exactly as approved and detect unauthorized or unexpected changes after deployment.

Approved state vs. Runtime state -> Detect drift

End-to-End Protection Flow

Trusted Packages -> Locked Dependencies -> SBOM -> Trusted Models & Data -> Secure CI/CD -> Verified Containers -> Controlled Builds -> Traceable Releases -> Runtime Monitoring

GOAL

Trust and verify every component from development to production.